SITE
DU BROCÉLIANDE TRIATHLON

Ce dossier contient le site du club Brocéliande Triathlon (Montfort-sur-Meu). Ce guide s'adresse aux bénévoles, même sans expérience technique.

Tu gères le site ? Le **guide de gestion** (fichier `guides/GUIDE_GESTION.md`) explique la publication sur GitHub et les mises à jour à faire au fil de la saison, fichier par fichier. Le **guide du code** (fichier `guides/GUIDE_CODE.md`) (aussi en PDF dans le dossier `guides/`) explique les bases du HTML et de la gestion d'un site, puis comment nos pages sont écrites, pour modifier un texte, ajouter une section ou une page.

Le principe. Les textes et les listes (partenaires, articles de presse, agenda, tarifs…) se trouvent dans de petits fichiers faciles à modifier. À chaque modification, GitHub fabrique le site complet et le met en ligne tout seul.

Rien à installer. Tout se gère directement sur github.com, avec un navigateur : on ouvre un fichier, on clique sur le crayon, on enregistre (guide de gestion, section 4). Les outils décrits aux sections 1, 2 et 12 sont facultatifs : ils servent à voir le site sur son ordinateur avant de publier, et à quelques tâches rares (PDF de la check-list et des guides, fichier de relecture).

Version du 30 octobre 2026 · Source : README.md, PDF refait avec npm run guide

SOMMAIRE

1. Installer le projet sur son ordinateur (facultatif)
2. Voir le site sur son ordinateur (facultatif)
3. Modifier un contenu
4. Exemple : ajouter un article de presse
5. Exemple : ajouter un partenaire
6. Remplacer une photo
7. Générer le site
8. Mettre le site en ligne
9. Le nom de domaine broceliandetri.org
10. Formulaire de contact
11. Organisation du dossier
12. Pour aller plus loin

1. INSTALLER LE PROJET SUR SON ORDINATEUR (FACULTATIF)

Cette étape n'est utile que pour travailler sur son ordinateur (voir le guide de gestion, section 3.8). Pour gérer le site sur github.com, passe directement à la section 3.

1. **Installer Node.js.** Va sur nodejs.org et télécharge la version « LTS ». Lance l'installation en gardant les choix proposés.
2. **Ouvrir un terminal dans le dossier du site.**
   - Sous Windows : ouvre le dossier dans l'Explorateur, clique dans la barre d'adresse, tape `cmd` puis Entrée.
   - Sur Mac : clic droit sur le dossier, puis « Nouveau terminal au dossier ».
3. **Installer les outils du site.** Tape la commande suivante, puis Entrée. Cela prend une à deux minutes.

```
npm install
```

C'est tout. Les fois suivantes, tu passes directement à l'étape 2.

2. VOIR LE SITE SUR SON ORDINATEUR (FACULTATIF)

Dans le terminal, ouvert dans le dossier du site :

```
npm run dev
```

Ouvre ensuite http://localhost:8080 dans ton navigateur. Chaque fois que tu enregistres un fichier, la page se met à jour toute seule.

Pour arrêter, reviens dans le terminal et appuie sur `Ctrl` + `C`.

3. MODIFIER UN CONTENU

Presque tous les contenus sont dans le dossier `src/_data/`, un fichier par sujet :

Fichier	Contenu
`actus.json`	Les cartes « L'actu fraîche du BT » de l'accueil
`presse.json`	La revue de presse (page Vie du club)
`publications.json`	Les vignettes Facebook et Instagram
`agenda.json`	L'agenda du club (frises de l'accueil et de la page Vie du club)
`checklist.json`	La check-list du triathlète (PDF à imprimer et pense-bête de la page Nous rejoindre)
`evenements.json`	Le triathlon et le biathlon : dates, textes, horaires, boutons, affiches
`resultats.json`	Les archives des résultats (liens Klikego)
`tarifs.json`	Les formules d'adhésion
`documents.json`	Les documents à télécharger (page Nous rejoindre)
`bureau.json`	Le bureau et les missions des bénévoles
`tenues.json`	L'équipement du club, par discipline (frise de losanges de la page Le club)
`disciplines.json`	Les 4 cartes Natation, Vélo, Course à pied, PPG
`semaine.json`	La semaine type (les séances et leurs horaires, pour chaque saison)
`lieux.json`	Les lieux d'entraînement et de contact
`partenaires.json`	Les partenaires principaux et la liste des autres partenaires
`collectivites.json`	Les collectivités qui soutiennent le club
`club.json`	Les chiffres clés (création, nombre de licenciés…)
`reseaux.json`	Les liens Instagram, Facebook, YouTube et Strava
`site.json`	E-mail, adresse, adresse du site, réglage du formulaire de contact
`legal.json`	Les informations des pages Mentions légales et Confidentialité : association, n° RNA, siège, directeur de la publication, hébergeur, contact pour les données

Les textes propres à une page (titres, paragraphes) sont dans les fichiers `.njk` du dossier `src/`, par exemple `src/le-club.njk`. Modifie seulement le texte entre les balises (`<p>…</p>`), sans toucher aux balises elles-mêmes.

Pour modifier ces fichiers, le plus simple est de le faire directement sur github.com : ouvre le fichier, clique sur le crayon, modifie, puis **Commit changes** (guide de gestion, section 4). Sur ton ordinateur, utilise un éditeur de texte simple, comme Visual Studio Code (gratuit), le Bloc-notes ou TextEdit. Évite Word.

Les règles des fichiers `.json` :

- Le texte s'écrit entre guillemets droits : `"comme ceci"`. Les accents sont acceptés.
- Les éléments d'une liste sont séparés par une virgule, **sauf après le dernier**.
- Pour écrire des guillemets dans un texte, utilise les guillemets français « » ou l'apostrophe.
- En cas d'erreur, la publication s'arrête (croix rouge dans l'onglet Actions du dépôt) et l'ancienne version reste en ligne. Le message indique le nom du fichier et la ligne : il s'agit presque toujours d'une virgule ou d'un guillemet oublié. Sur ton ordinateur, le terminal affiche le même message.

Une information encore inconnue peut s'écrire provisoirement entre crochets, par exemple `[date]` : elle s'affiche telle quelle sur le site. Le journal de chaque publication (onglet Actions, étape « Repérer les textes provisoires entre crochets ») liste celles qui restent ; sur ton ordinateur, `npm run build` puis `npm run crochets` fait la même chose.

4. EXEMPLE : AJOUTER UN ARTICLE DE PRESSE

1. Ouvre `src/_data/presse.json`.
2. Le fichier contient une liste d'articles. Chaque article est un bloc entre accolades `{ … }`. Pour qu'il apparaisse en premier, ajoute ton bloc tout en haut, juste après le crochet `[`. Termine-le par une virgule :

```
[
  {
    "source": "Ouest-France",
    "date": "Mars 2027",
    "titre": "Titre exact de l'article",
    "url": "https://www.ouest-france.fr/adresse-de-l-article",
    "couleur": "rose"
  },
  {
    "source": "Ouest-France",
    "date": "Juin 2026",
    …
```

3. `couleur` est la couleur du trait au-dessus de la carte : `rose`, `orange` ou `noir`.
4. Enregistre : **Commit changes** sur github.com. Deux à trois minutes plus tard, la page Vie du club en ligne est à jour. (Sur ton ordinateur, si `npm run dev` tourne, la page se met à jour aussitôt.)

Pour qu'un article apparaisse aussi dans « L'actu fraîche » de l'accueil, ajoute-le de la même façon dans `actus.json`, avec une photo (voir la section 6). Les cartes de l'accueil indiquent en plus `categorie` (le texte de l'étiquette sur la photo), `couleurCategorie` (`noir` ou `rose`), `image` (nom du fichier photo) et `alt` (description de la photo pour les personnes aveugles).

5. EXEMPLE : AJOUTER UN PARTENAIRE

1. **Le logo.** Envoie le logo dans le dossier `src/assets/img/` (sur github.com : **Add file** → **Upload files**), de préférence en PNG ou en JPG. Donne-lui un nom simple, sans espace ni accent, par exemple `sp_monpartenaire.png`.
2. **La fiche.** Ouvre `src/_data/partenaires.json` et ajoute un bloc dans la liste `principaux` :

```
{
  "nom": "Nom du partenaire",
  "logo": "sp_monpartenaire.png",
  "activite": "Son activité",
  "commune": "Sa commune",
  "description": "Une ou deux phrases de présentation.",
  "url": "https://www.site-du-partenaire.fr/",
  "texteLien": "site-du-partenaire.fr"
},
```

3. Sans logo pour l'instant, laisse `"logo": ""`. La carte affiche alors le nom avec la mention « [logo à ajouter] ».
4. Pour un partenaire de la liste « Merci à tous nos autres partenaires », ajoute simplement son nom, entre guillemets, dans la liste `autres`.

6. REMPLACER UNE PHOTO

Les photos et logos se trouvent dans `src/assets/img/`.

Sur github.com, ouvre le dossier, puis **Add file** → **Upload files** et glisse la photo (guide de gestion, section 4.2).

- **Garder le même nom de fichier** : envoie la nouvelle photo avec exactement le même nom. Elle remplace l'ancienne, et le site l'utilise automatiquement.
- **Utiliser un nouveau nom** : envoie d'abord la photo dans `src/assets/img/`, puis indique son nom dans le fichier `.json` concerné (champ `image` ou `logo`). Pense à mettre à jour le champ `alt`, qui décrit la photo en une phrase.

Conseils :

- Préfère le JPG pour les photos et le PNG pour les logos.
- 2 000 pixels de large suffisent. Le site crée tout seul des versions plus légères (AVIF et WebP) pour un affichage rapide.
- Le champ `cadrage` (par exemple `"50% 30%"`) choisit la partie de la photo qui reste visible quand elle est recadrée. Le premier nombre règle la position horizontale, le second la position verticale. Laisse `""` pour centrer.

7. GÉNÉRER LE SITE

Sur GitHub, c'est automatique : à chaque modification enregistrée, le site est vérifié (HTML valide, liens internes), fabriqué, puis mis en ligne. Le fichier qui commande tout cela est `.github/workflows/publier.yml`.

Sur ton ordinateur (facultatif) :

```
npm run build
```

Le site complet est créé dans le dossier `dist/`. C'est ce dossier, et lui seul, qui serait à envoyer chez un autre hébergeur que GitHub Pages.

Tu peux aussi lancer une vérification complète, plus poussée que celle de GitHub :

```
npm run check
```

Elle contrôle le code HTML, vérifie que tous les liens internes fonctionnent et teste l'accessibilité de chaque page sur ordinateur et sur mobile. Elle a besoin de Google Chrome ou de Microsoft Edge, qui sont déjà installés sur la plupart des ordinateurs.

8. METTRE LE SITE EN LIGNE

Le site est fait de simples fichiers, sans base de données. Il peut donc être hébergé presque partout. **Le choix retenu est GitHub Pages** : tout est prêt, et la démarche complète est dans le **guide de gestion** (fichier `guides/GUIDE_GESTION.md`), section 3. Les deux autres possibilités restent ouvertes :

Hébergeur	Prix	Comment ça marche
Netlify	Offre gratuite	Glisser-déposer le dossier `dist/` sur le site de Netlify, ou le relier à un dépôt GitHub pour une mise à jour automatique. Propose aussi un service de formulaires.
GitHub Pages	Gratuit	Le projet est déposé sur GitHub et le site est publié à chaque modification. La publication automatique est déjà prête (`.github/workflows/publier.yml`) : voir le **guide de gestion** (fichier `guides/GUIDE_GESTION.md`).
OVH (ou autre hébergeur web)	Payant, offres d'entrée de gamme peu chères	On envoie le contenu du dossier `dist/` par FTP, avec un logiciel comme FileZilla.

Dans tous les cas, la page d'erreur `404.html` est fournie : la plupart des hébergeurs l'utilisent automatiquement.

9. LE NOM DE DOMAINE BROCELIANDETRI.ORG

Le site actuel du club est hébergé chez Wix, à l'adresse `broceliandetri.org`. Pour que cette adresse affiche le nouveau site, il faudra reprendre la main sur le nom de domaine :

- soit le **faire pointer** vers le nouvel hébergeur (réglage DNS chez le gestionnaire du domaine) ;
- soit le **transférer** chez un autre gestionnaire, s'il a été acheté via Wix.

C'est une décision du bureau, à prendre avant la mise en ligne. Les adresses e-mail en `@gmail.com` ne sont pas concernées.

Si l'adresse du site change, mets-la à jour dans `src/_data/site.json` (champ `url`).

10. FORMULAIRE DE CONTACT

Le site ne possède pas de serveur. Pour l'instant, le bouton « Envoyer » ouvre la messagerie du visiteur avec le message prérempli, adressé à `broceliandetriathlon@gmail.com`.

Pour recevoir les messages directement, sans passer par la messagerie du visiteur, voici des services gratuits (aucun n'est activé) :

- **Netlify Forms**, si le site est hébergé chez Netlify ;
- **Formspree**, **Web3Forms** ou **FormSubmit**, qui fonctionnent avec n'importe quel hébergeur.

Leurs offres gratuites limitent le nombre de messages par mois. Une fois le service choisi, il suffit généralement de remplacer l'adresse `action` dans `src/_data/site.json` (et `encodage` par `application/x-www-form-urlencoded`), en suivant les instructions du service. Le champ anti-spam caché (`site_web`) est déjà en place.

Pense aussi à compléter la page Confidentialité (`src/confidentialite.njk`) avec le service retenu.

11. ORGANISATION DU DOSSIER

Dossier ou fichier	Contenu
`src/`	Le site : pages (`.njk`), données (`_data/*.json`), briques communes (`_includes/`), images, PDF, vidéo, CSS et JavaScript (`assets/`). C'est le seul dossier qu'on modifie au quotidien
`guides/`	Le guide de gestion et le guide du code, en Markdown et en PDF
`scripts/`	Les outils lancés par les commandes `npm run …` (contrôles, PDF de la check-list et des guides, recadrage des tenues, fichier de relecture)
`sources/`	La source de la vidéo d'ouverture (GIF)
`partage/versions.json`	L'historique des versions de relecture (numéro, date, contenu)
`.github/workflows/publier.yml`	La publication automatique sur GitHub Pages (vérification, recadrage des tenues, fabrication, mise en ligne)
`dist/` , `node_modules/` , `.cache/`	Fabriqués par les outils, jamais envoyés sur GitHub

12. POUR ALLER PLUS LOIN

Ces commandes se lancent dans un terminal, sur un ordinateur équipé de Node.js (section 1). Aucune n'est nécessaire pour gérer le site sur github.com.

Commande	Rôle
`npm run dev`	Affiche le site sur l'ordinateur, avec mise à jour automatique
`npm run build`	Génère le site dans `dist/`
`npm run check`	Vérifie le HTML, les liens internes et l'accessibilité
`npm run crochets`	Liste les informations encore entre crochets (après `npm run build`)
`npm run audit`	Note chaque page avec Lighthouse : vitesse, accessibilité, bonnes pratiques, référencement (connexion internet nécessaire)
`npm run clean`	Vide le dossier `dist/`
`npm run html`	Crée un seul fichier HTML qui contient tout le site (pages, photos, polices, vidéo, PDF), dans `LIVRABLES/3_VERSIONS_RELECTURE/`. Il s'ouvre d'un double-clic et s'envoie par e-mail ou messagerie pour la relecture (connexion internet nécessaire pour Facebook, Instagram et la vidéo)
`npm run tenues`	Recadre les visuels des tenues (`src/_data/tenues.json`) pour que chaque article tienne entier dans son losange. La publication sur GitHub le fait toute seule : la commande sert à voir le résultat avant
`npm run guide`	Refait les PDF des guides (`guides/GUIDE_GESTION.pdf`, `guides/GUIDE_CODE.pdf`, `guides/README.pdf`) à partir des fichiers `.md`. À lancer après chaque modification d'un guide
`npm run checklist`	Recrée le PDF à imprimer de la check-list du triathlète (`src/assets/docs/check-list-triathlete.pdf`) à partir de `src/_data/checklist.json`
`npm run video`	Recrée la vidéo d'ouverture à partir de `sources/ouverture_loop.gif`. À utiliser seulement si le GIF change, et nécessite ffmpeg

Bon à savoir :

- **Le compte à rebours** « J-xxx » se calcule à partir de `triathlon.depart` dans `src/_data/evenements.json`. Pour la prochaine édition, change seulement cette date.
- **La vidéo d'ouverture** ne se lance pas si le visiteur a demandé à son appareil de réduire les animations. Dans ce cas, seule l'image fixe s'affiche.

- **Aucun cookie, traceur ni outil de statistiques** n'est utilisé par le site. Les polices sont hébergées avec le site : aucun appel à Google Fonts.
- **Facebook et Instagram** (accueil et Vie du club). Les publications Facebook et Instagram s'affichent avec les codes d'intégration officiels de Meta. Meta dépose des cookies : ces contenus ne sont donc chargés qu'avec l'accord du visiteur (règles de la CNIL).
  - Un seul accord couvre les deux réseaux. Il est demandé par un bandeau (« Refuser » / « Accepter »), sur les pages concernées uniquement.
  - Le choix est gardé 6 mois. Le visiteur peut le retirer sous les contenus, ou depuis la page Confidentialité.
  - Sans accord, le visiteur voit les vignettes de `publications.json`.
  - Ce sont des publications choisies, qui se règlent dans `publications.json` (`facebookIntegrations` et `instagramIntegrations`). Instagram ne propose pas de fil automatique, et le fil de la page Facebook (« plugin Page ») n'affiche plus les publications aux visiteurs non connectés à Facebook.
  - Pour désactiver l'un ou l'autre, mets `"filFacebook": false` ou `"integrationInstagram": false` dans `src/_data/site.json`.
- **La check-list du triathlète** existe en deux versions tirées du même fichier `src/_data/checklist.json` : le PDF à imprimer (`npm run checklist`, nécessite Chrome ou Edge) et le pense-bête de la page Nous rejoindre.
- **Les couleurs et les polices** sont réglées au début de `src/assets/css/main.css`.